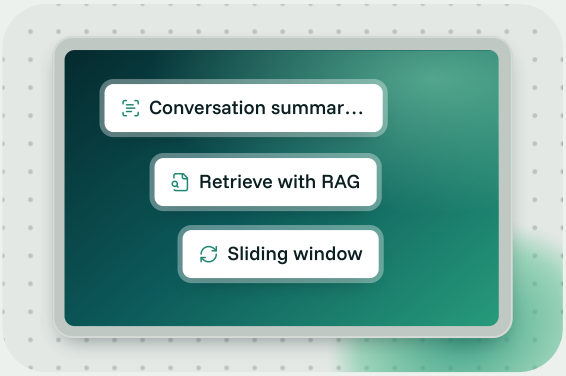
Last updated August 2026

Context Windows

How much text a language model can hold in mind at once, why bigger is not always better, and what happens when you run out of room.



12 min read · Beginner-friendly with advanced sections

Pascal Academy
AI fundamentals · Part 5 of 10

What Is a Context Window?

In the first guide, we established that a language model predicts the next token based on the tokens that came before it. But the model cannot look back infinitely. There is a ceiling on how many tokens it can consider at once, and that ceiling is called the context window.

The best way to think of it is as the model's working memory. When you paste a long document into a chat, the model reads it into its context window. When you have a multi-turn conversation, each earlier message stays in the window so the model can reference what was said. The context window is the total space available for everything the model needs to hold in mind to produce its next response.

A model with a 4,000-token context window can hold roughly 3,000 words of combined input and output. A model with a 128,000-token window can hold about 96,000 words, enough for a short book. The difference in what you can do with those two models is enormous. The 4,000-token model chokes on a research paper. The 128,000-token model can read several papers and compare them in a single response.

Key principle

The context window includes everything the model processes in a single call. That means the system prompt, the user's message, any retrieved documents, the conversation history, and the model's own response all count against the same budget. A 128,000-token window covers 128,000 tokens for everything combined.

What Fills the Context Window

People often assume the context window is the size of their question. In practice, multiple things consume tokens before the model generates a single word of response.

Component	What It Is	Typical Token Cost
System prompt	Instructions that define the model's role, behaviour, and constraints for the entire conversation. Often hidden from the user in chat applications.	200 to 2,000 tokens depending on complexity.
Conversation history	All prior user and assistant messages in the current session. Grows with every turn and is the main reason long conversations eventually hit the ceiling.	50 to 500 tokens per turn, accumulating rapidly.
Retrieved documents	External content fetched by a RAG system and inserted into the prompt so the model can ground its answer in specific sources.	500 to 10,000 tokens depending on how many chunks are retrieved.
User message	The actual question or instruction from the user for this turn.	10 to 5,000 tokens depending on whether the user pasted content.
Model output	The response the model generates. This also counts against the context window, reducing what is available for input.	100 to 4,000 tokens depending on the max output cap.

The practical implication is that a model with a 128,000-token context window does not let you paste 128,000 tokens of input. After the system prompt, conversation history, and a safety margin for the model's response, you might have 120,000 tokens of usable input space. In a multi-turn conversation, each turn eats into that budget, and eventually the oldest messages need to be dropped or summarised to make room for new ones.

Why There Is a Limit

The context window is finite because of how self-attention works. As we covered in the first guide, self-attention lets every token in the input look at every other token to determine relevance. This is what gives Transformers their power, but it comes with a computational cost that grows quadratically.

Quadratic growth means that doubling the number of tokens in the context window does not double the compute. It quadruples it. A model processing 1,000 tokens computes 1 million attention relationships. At 2,000 tokens, that becomes 4 million. At 10,000 tokens, 100 million. This is why early models had small context windows. The hardware could not handle the computation that larger windows demanded.

Recent years have seen dramatic improvements. Techniques like rotary position embedding (RoPE), sparse attention, and flash attention have reduced the compute penalty of long contexts. Hardware has gotten faster. These advances are why a model in 2022 had a 4,000-token window and a comparable model in 2025 can handle 128,000 or even a million tokens. The fundamental quadratic scaling has not gone away, but it has been mitigated enough to make large context windows practical.



Go deeper

Why attention is quadratic

Optional

Self-attention computes a relationship between every pair of tokens in the sequence. For N tokens, that means N times N comparisons, stored in a matrix of size N-squared. If you double N, the matrix becomes four times larger. This is the core bottleneck. Modern techniques like flash attention do not change the number of comparisons, but they reduce the memory bandwidth needed to move the data through the GPU, which is often the actual bottleneck in practice. Sparse attention skips some comparisons entirely, trading a small amount of accuracy for a large reduction in compute.

The Lost-in-the-Middle Problem

A larger context window lets you put more text in, but it does not guarantee the model uses all of that text equally. Research published in 2023 by Liu et al. found that language models pay uneven attention across long inputs. Information at the beginning and end of the context gets more weight than information in the middle. The phenomenon is called the lost-in-the-middle effect, and it has practical consequences for anyone building LLM applications.

If you paste a 50-page document into a prompt and ask the model to find a specific detail, the model is more likely to find it if it appears on the first or last page. If the detail is on page 27, the model may miss it entirely, even though the document is well within the context window. The information is there. The model just does not attend to it as strongly.

This finding has direct implications for prompt design and RAG systems. If you are retrieving documents and placing them in the context, put the most relevant ones at the beginning and end of the context block, not in the middle. If you are writing instructions, put the most important ones at the top of the prompt and repeat them just before the model starts generating. The lost-in-the-middle effect is also an argument for keeping context lean. Fewer tokens mean the model's attention is more concentrated, and the probability of missing something important goes down.



Pascal Academy

Pascal Academy's prompt engineering workshop covers practical strategies for mitigating the lost-in-the-middle effect, including context ordering, re-ranking, and prompt structure patterns. Reach out to learn about team training options.

Ask about team training

Cost and Latency Trade-offs

Larger context windows cost more money and take more time. Both effects are linear in token count for most API pricing models, but the latency effect also has a quadratic component because of the attention computation.

On the cost side, API providers charge per token for both input and output. A prompt with 50,000 tokens of retrieved documents costs roughly 50 times more per call than a prompt with 1,000 tokens. For an application making thousands of calls per day, the difference between a 1,000-token prompt and a 50,000-token prompt can be the difference between a manageable bill and an eye-watering one. This is why RAG systems are designed to retrieve only the relevant chunks rather than dumping the entire knowledge base into the context.

On the latency side, the model takes longer to produce its first token when the context is large. This is called time-to-first-token, and it matters for user experience. A chatbot that takes 8 seconds to start responding because it is processing a 100,000-token context feels broken. For interactive applications, keeping the context lean is also a user experience decision.

The relationship between context size and latency is not strictly linear. The initial processing of the input, called prefill, scales with the number of tokens. But once the model starts generating output, each new token only needs to attend to the already-processed context, which is faster. So the first token is slow and subsequent tokens are faster. The longer the context, the longer the wait for that first token.

Factor	How It Scales	Practical Impact
API cost	Linear with token count. More input tokens means a higher per-call charge.	A 50,000-token prompt costs 50x more than a 1,000-token prompt at the same price per token.
Time to first token	Roughly linear with a quadratic component from attention. More context means a longer wait before the model starts generating.	A 100,000-token context might take 5 to 10 seconds to prefill. A 1,000-token context responds in under a second.
Memory usage	Linear with token count. The model stores key-value pairs for every token in the context.	Large contexts consume GPU memory, which limits how many concurrent requests a single server can handle.
Attention quality	Degrades with context length due to the lost-in-the-middle effect.	A model with a 128,000-token window may perform worse on retrieval tasks than a model with a 4,000-token window if the answer is buried in the middle.

Context Window Sizes Across Models

Context windows have grown dramatically since ChatGPT launched in late 2022 with a 4,000-token limit. The table below shows how far the field has come and where the major models stand as of mid-2026.

Model	Context Window	Approximate Word Equivalent
GPT-3.5 (original ChatGPT, 2022)	4,000 tokens	~3,000 words (about 6 pages)
GPT-4 (launch, 2023)	8,000 tokens	~6,000 words (about 12 pages)
GPT-4o (2024)	128,000 tokens	~96,000 words (about a short book)
Claude 3.5 Sonnet (Anthropic)	200,000 tokens	~150,000 words (a full novel)
Gemini 1.5 Pro (Google)	2,000,000 tokens	~1.5 million words (about 15 books)
Llama 3.1 (Meta)	128,000 tokens	~96,000 words (about a short book)

The jump from 4,000 to 2 million tokens in under three years represents a 500x increase. Whether this trend continues depends on whether the computational challenges of quadratic attention can be solved at scale, or whether alternative architectures replace the Transformer entirely. For now, the practical question is not whether you have enough context, but whether you are using the context you have wisely.

Go deeper

Token efficiency varies by language

Optional

The word-to-token ratio is roughly 1.5 tokens per word for English, but this varies significantly across languages. Languages with non-Latin scripts, including Hindi, Tamil, Telugu, and Bengali, often produce more tokens per word because their tokenizers were optimised primarily for English text. A sentence in Telugu can produce 7 times more tokens than the same sentence in English. This means a 128,000-token context window holds far less Telugu text than English text, which has real implications for multilingual applications and for cost when serving non-English users.

Managing Context Limits

Even with large context windows, you need strategies for managing what goes into them. The goal is to maximise the signal the model receives while minimising the cost and latency of processing it.

Pascal Academy

Pascal Academy's RAG engineering course covers context management in depth, including when to use RAG versus long context, conversation summarisation patterns, and cost optimisation. Reach out to learn about cohort programmes for your engineering team.

Ask about team training

Conversation Summarisation

In a long multi-turn conversation, early messages eventually consume too much context. A common strategy is to summarise older messages once they exceed a certain age. The model or a separate pipeline compresses the first N messages into a summary, and the summary replaces the original messages in the context. The conversation continues with the summary providing a condensed memory of what was discussed, while recent messages remain in full.

The trade-off is that summarisation loses detail. A summary of a 20-message exchange will not capture every nuance, and the model may miss references to specific things said earlier. For applications where precision matters, an alternative is selective retention, keeping only the messages that are relevant to the current topic and dropping the rest.

Retrieve with RAG

As covered in the RAG guide, retrieving only the relevant chunks from a knowledge base is far more efficient than dumping the entire database into the context. RAG keeps the context lean by filtering at the retrieval stage, so the model sees only the passages likely to contain the answer. This reduces cost, improves latency, and mitigates the lost-in-the-middle effect by keeping the context small enough for the model to attend to everything.

The question of when to use RAG versus long context is a practical one. If your knowledge base is under 100,000 tokens and rarely changes, putting the whole thing in the context is simpler and avoids the infrastructure of a vector database. If the knowledge base is large or changes frequently, RAG is the right approach. Many production systems use both, putting a small set of retrieved documents in the context alongside a concise system prompt.

Sliding Window

A sliding window keeps only the most recent N tokens in context and drops everything older. This is the simplest approach and works for applications where recent context is what matters. Chatbots that only need to remember the last few exchanges can use a sliding window without any summarisation overhead. The limitation is that anything beyond the window is forgotten entirely, which breaks down for tasks that reference earlier context.

Security Implications of Long Context

A larger context window gives the model more text to work with, but it also gives an attacker more surface area to exploit. Research from Anthropic demonstrated that models with longer context windows are more vulnerable to jailbreaking attacks, where adversarial text buried deep in the context attempts to override the model's safety instructions.

The mechanism is straightforward. In a long context, the model's attention is spread across many tokens. Safety instructions in the system prompt get diluted when thousands of tokens of user-provided content follow them. An attacker can embed instructions in a retrieved document, a pasted article, or a long conversation history that steer the model toward producing harmful output. The larger the context, the more room there is to hide such instructions.

Mitigations include keeping contexts as small as possible, using delimiters to separate instructions from data (as covered in the prompt engineering guide), treating all user-provided content as untrusted, and monitoring model output for signs of successful injection. No single mitigation is sufficient. Defense in depth, combining prompt-level controls with output filtering and tool-level permissions, is necessary for any system processing untrusted input.

Practical Guidelines

Based on everything covered so far, here are practical recommendations for working with context windows in production.

Situation	Recommendation
Analysing a single document under 100,000 tokens	Put the document directly in the context. Simpler than RAG and avoids retrieval errors. Place instructions after the document to mitigate the lost-in-the-middle effect.
Answering questions from a large, frequently updated knowledge base	Use RAG. Retrieve 3 to 5 relevant chunks per query. Keep the total context under 10,000 tokens for cost and latency.
Long multi-turn conversations	Implement conversation summarisation once the history exceeds half the context window. Keep the last 5 to 10 turns in full, summarise the rest.
Interactive applications needing fast response	Minimise context size. Target under 4,000 tokens of input for sub-second first-token latency. Use RAG with a small top-K rather than dumping large documents.
Handling untrusted user content	Use delimiters to separate user content from instructions. Keep context as small as possible. Monitor output for signs of prompt injection. Never put secrets in the system prompt.

The Future of Context Windows

Context windows have grown 500x in three years, and there is active research on pushing them further. Google's Gemini 1.5 Pro already handles 2 million tokens, and experimental models have demonstrated effective attention over even longer sequences. Whether this growth continues depends on whether the quadratic scaling of attention can be circumvented at scale.

Several research directions are promising. Sparse attention mechanisms skip irrelevant token pairs entirely, reducing the compute from quadratic to near-linear for long sequences. Alternative architectures like Mamba and state space models replace the attention mechanism with a fixed-size recurrent state, which does not grow with sequence length at all. If these architectures prove as capable as Transformers for language tasks, the context window constraint may eventually disappear.

For now, the context window remains a practical constraint that shapes how LLM applications are built. The strategies in this guide, from RAG to summarisation to careful prompt ordering, exist because the window is finite. Understanding the constraint and designing around it is a core skill for anyone building with language models.

Summary

A context window is the maximum number of tokens a model can process in a single call, covering the system prompt, conversation history, retrieved documents, user message, and model output combined. The window is finite because self-attention scales quadratically with sequence length, though modern techniques have pushed windows from 4,000 tokens to over 2 million. Larger windows cost more, run slower, and are vulnerable to the lost-in-the-middle effect where the model ignores information in the middle of long contexts. Managing context well means keeping it lean through RAG, summarisation, and selective retention, and ordering information so the most important content is at the beginning and end. Security risks grow with context size because adversarial instructions have more room to hide.

About this guide

This guide is part of Pascal Academy's AI Fundamentals series, covering LLMs, prompt engineering, RAG, agents, and context windows. The full series and hands-on courses are available at Pascal Academy. For teams looking to upskill, we offer custom cohort programmes tailored to your stack and use cases.

Frequently asked questions

What is a context window in AI?

A context window is the amount of text an AI model can process in a single interaction. It includes the prompt, any retrieved documents, and the response. Everything the model

sees must fit within this limit. Larger context windows allow longer documents but cost more and can introduce latency.

Why do context windows have a limit?

The attention mechanism that powers language models scales quadratically with input length. Doubling the context window roughly quadruples the compute required. This creates a practical trade-off between how much text the model can process and how fast and cheap it is to run.

How does Pascal handle context windows for long financial documents?

Pascal's agents are designed to work with long filings and research documents by using retrieval and chunking strategies. Instead of stuffing an entire annual report into a single context window, the agents retrieve the relevant sections, process them, and synthesize results with citations. This keeps outputs accurate and auditable.

What is lost-in-the-middle in AI models?

Lost-in-the-middle is a phenomenon where models pay less attention to information in the middle of a long context window. They tend to focus on the beginning and end of the input. This matters for finance because critical data in the middle of a filing can be overlooked if the context window is too large and the prompt is not structured well.